AI Implementation Prompt for Foto AI Agent

Instructions for Using This Prompt

Copy the entire prompt below and paste it into any AI assistant (ChatGPT, Claude, Gemini, etc.) along with the `FOTO_AI_AGENT_IMPLEMENTATION_SPEC.md` file to have it implement the Foto AI Agent system.

PROMPT START

I need you to implement a **Foto AI Agent** - an autonomous photography coaching system based on the detailed specification I'm providing.

Project Context

Current State:
- Next.js web application located at `/home/ubuntu/foto_app/nextjs_space/`
- Existing chatbot-based photography coach (simple iframe embed)
- Basic website with home, about, and chat pages
- Deployed at: https://myfoto.topues.com

Goal:
Transform this into a fully autonomous AI Agent that:
- Proactively coaches users based on their schedule
- Sends context-aware daily photography challenges
- Adapts to work schedule, weather, and user patterns
- Tracks progress and skill development
- Works 24/7 in the background

What I'm Providing

I have a **complete implementation specification** (attached: `FOTO_AI_AGENT_IMPLEMENTATION_SPEC.md`) that contains:

1. ✅ Complete technical architecture
2. ✅ Full database schema (Prisma models)
3. ✅ AI agent decision logic (with code)
4. ✅ All challenge types and specifications
5. ✅ Notification strategies and email templates
6. ✅ External API integrations (Weather, Email, Calendar)
7. ✅ UI component specifications
8. ✅ Scheduled task implementations
9. ✅ API endpoint definitions
10. ✅ Step-by-step implementation guide

Please read the specification document thoroughly before starting.

What's Already Done

The foundation is set up:
- ✅ PostgreSQL database initialized
- ✅ Prisma schema created and pushed to database
- ✅ NextAuth authentication configured
- ✅ LLM APIs (Abacus.AI) configured
- ✅ Project structure exists

What You Need to Build

Phase 1: Core Backend (High Priority)

1. **Authentication System**
   - Create signup/login pages using NextAuth
   - Implement user profile management
   - Create onboarding flow (wizard to set preferences)
   - Default test account: sesal78@gmail.com / foto2024

2. **Database Seeding**
   - Create seed script (`/scripts/seed.ts`)
   - Seed test user with:
     - Profile (name: Seun, email: sesal78@gmail.com)
     - Preferences (Saturday primary, weekday lunch enabled, etc.)
     - Gear (Fujifilm X-T5, Ricoh GR IIIx, Nikon FE2, Pentax ME)
     - Sample challenges (5-10 past challenges with various statuses)
     - Progress stats

3. **Challenge Generation System**
   - Create `/lib/challenge-generator.ts`
   - Implement the AI Agent decision logic from the spec (Section 6)
   - Context analyzer: detect day type, weather, WFH status
   - Challenge generator: use Foto ChatLLM API
   - Weather adaptation logic
   - Adaptive difficulty adjustment

4. **API Endpoints**
   Implement these critical APIs:
   - `POST /api/auth/signup` - User registration
   - `GET /api/user/profile` - Get user profile
   - `PUT /api/user/preferences` - Update preferences
   - `GET /api/challenges` - List challenges (with filters)
   - `GET /api/challenges/:id` - Get challenge details
   - `POST /api/challenges/:id/complete` - Mark complete (with photo upload)
   - `POST /api/challenges/:id/skip` - Skip challenge
   - `GET /api/progress` - Get progress stats
   - `POST /api/challenges/generate` - Manual challenge generation (for testing)

5. **External Integrations**
   - **Weather API**: OpenWeatherMap integration
     - Get current weather
     - Get forecast
     - Detect special conditions (fog, clear skies, etc.)
     - **Email Service**: SendGrid integration
     - Send challenge emails (use templates from spec)
     - Send alerts
     - Send weekly reports
     - **Google Calendar** (optional but recommended):
     - OAuth setup
     - Detect WFH days
     - Add photo walk events

Phase 2: User Interface (High Priority)

1. **Authentication Pages**
   - `/signup` - Registration page
   - `/login` - Login page
   - Onboarding wizard (first-time user setup)

2. **Main Dashboard** (`/dashboard`)
   - Today's challenge card (prominent)
   - Stats widgets (streak, completion rate, total)
   - Upcoming challenges preview
   - Recent submissions gallery
   - Weather widget
   - Quick actions (mark complete, skip, upload)

3. **Challenges Page** (`/dashboard/challenges`)
   - List view with tabs: Pending, Completed, Skipped
   - Filter by type, date, category
   - Challenge detail modal/page
   - Complete challenge flow
   - Upload photo form
   - AI feedback display

4. **Progress Page** (`/dashboard/progress`)
   - Overall stats (streak, completion rate)
   - Completion rates by type (bar charts)
   - Skill progression display
   - Favorite subjects (tags)
   - Improvement areas
   - Download report button

5. **Settings Page** (`/dashboard/settings`)
   - Work schedule form
   - Photography availability settings
   - Challenge preferences
   - Notification preferences

- Gear management
- Integration toggles (Calendar, Location)
- Pause/reduce options

Phase 3: Scheduled Tasks (Critical for Autonomy)

Implement these scheduled tasks using the daemon management system:

1. **Daily Challenge Generator**
   - Saturday 7:00 AM → Weekend deep dive challenges
   - Sunday 8:00 AM → Light weekend challenges (optional)
   - Weekdays 11:00 AM → Lunch break micro-challenges
   - Weekday 9:00 AM (WFH days) → Flexible WFH challenges
   - Check user preferences before sending

2. **Weather Monitor**
   - Run every 3 hours
   - Check for special conditions (fog, clear skies, etc.)
   - Send golden hour alerts (1.5-2 hours before sunset)
   - Send weather opportunity alerts

3. **Weekly Report Generator**
   - Sunday 8:00 PM
   - Compile week's stats
   - Generate insights
   - Create PDF report
   - Send via email

4. **Calendar Sync** (if Calendar integration enabled)
   - Weekday mornings 6:00 AM
   - Check for WFH indicators in calendar
   - Adjust challenge type accordingly

5. **Streak Manager**
   - Daily 11:00 PM
   - Check if today's challenge was completed
   - Update streak (increment or reset)
   - Send milestone notifications (7, 30, 100 days)

Phase 4: Progress Tracking & Analytics

Implement these functions (refer to Section 12 of spec):

1. **Completion Rate Calculator**
   - Overall rate
   - Rate by challenge type
   - Rate by difficulty
   - Trend analysis

2. **Adaptive Difficulty System**
   - Monitor completion rates
   - Adjust challenge complexity automatically
   - Update user preferences based on patterns

3. **Skill Progression Analyzer**
   - Track completions by category
   - Calculate skill levels (beginner → advanced)
   - Detect improvement trends
   - Identify favorite subjects

Key User Context (CRITICAL)

The AI Agent MUST understand and adapt to this user profile:

```
{
  // Schedule
  workSchedule: "Monday-Friday 9am-5pm (office)",
  primaryPhotoDay: "Saturday (4-8 hours dedicated)",
  lunchBreaks: "20-30 mins (optional quick shots)",
  eveningPhotography: "Sometimes after work if golden hour",
  workFromHome: "Varies - detected from calendar",

  // Challenge Types by Time Available
  saturday: "WEEKEND_DEEP_DIVE (4-8 hrs, high complexity)",
  weekdayLunch: "LUNCH_MICRO (15-25 mins, single shot)",
  afterWork: "AFTER_WORK (45-90 mins, golden hour)",
  wfh: "WORK_FROM_HOME (30-60 mins, flexible timing)",

  // Gear
  mainCamera: "Fujifilm X-T5 (weekends, serious shoots)",
  everydayCamera: "Ricoh GR IIIx (lunch breaks, pocketable)",
  filmCameras: ["Nikon FE2", "Pentax ME"],

  // Interests
  likes: ["street", "portrait", "architecture"],
  skillLevel: "intermediate"
}
```

The agent must:
- ✅ Send complex challenges on Saturday (main photo day)
- ✅ Send simple micro-challenges on weekday lunch breaks
- ✅ Skip lunch challenges if weather is bad
- ✅ Send golden hour alerts if weather is perfect
- ✅ Adapt to work-from-home days (flexible timing)
- ✅ Respect that user has a day job (no disruptions during work hours)

Implementation Guidelines

1. Follow the Spec Structure

The specification document has 9 implementation phases. Follow them in order:
1. Foundation (Database & Auth)
2. Core API Endpoints
3. AI Challenge Generation
4. External Integrations
5. Dashboard UI
6. Scheduled Tasks
7. Progress Tracking

8. Testing
9. Deployment

2. Code Quality Standards

- **TypeScript**: Use strict typing throughout
- **Prisma**: Use Prisma Client for all database operations
- **Error Handling**: Comprehensive try-catch blocks
- **Validation**: Validate all user inputs
- **Comments**: Document complex logic
- **Security**: Never expose API keys, sanitize inputs

3. UI/UX Requirements

- **Responsive**: Mobile, tablet, desktop
- **Loading States**: Show loading indicators
- **Error States**: User-friendly error messages
- **Empty States**: Handle no data gracefully
- **Accessibility**: Proper ARIA labels, keyboard navigation
- **Design**: Clean, modern, photography-focused aesthetic

4. Testing Requirements

- Test authentication flow
- Test challenge generation for different contexts:
- Saturday morning
- Weekday lunch
- WFH day
- Bad weather day
- Golden hour conditions
- Test all CRUD operations
- Test scheduled tasks manually
- Test email delivery
- Test weather API integration

5. Environment Variables

Ensure these are set up:

```
DATABASE_URL="postgresql://..."
NEXTAUTH_SECRET="..."
NEXTAUTH_URL="..."
ABACUSAI_API_KEY="..." (already set)
OPENWEATHERMAP_API_KEY="..." (may already be configured)
SENDGRID_API_KEY="..." (needs to be configured)
```

6. Package Management

- **Always use `yarn`** (never npm)
- Install packages: `cd /home/ubuntu/foto_app/nextjs_space && yarn add <package>`
- Run commands: `yarn dev`, `yarn build`, `yarn prisma generate`, etc.

7. Project Structure

```
/home/ubuntu/foto_app/nextjs_space/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── signup/page.tsx
│   ├── dashboard/
│   │   ├── page.tsx (main dashboard)
│   │   ├── challenges/page.tsx
│   │   ├── progress/page.tsx
│   │   └── settings/page.tsx
│   ├── api/
│   │   ├── auth/[...nextauth]/route.ts
│   │   ├── challenges/route.ts
│   │   ├── user/profile/route.ts
│   │   └── ... (other endpoints)
│   └── layout.tsx (wrap with SessionProvider)
├── components/
│   ├── dashboard/
│   ├── challenges/
│   └── ui/ (already exists with shadcn components)
├── lib/
│   ├── challenge-generator.ts (AI agent logic)
│   ├── weather.ts (weather API integration)
│   ├── email.ts (SendGrid integration)
│   ├── calendar.ts (Google Calendar)
│   ├── progress.ts (analytics functions)
│   └── db.ts (Prisma client)
├── scripts/
│   └── seed.ts (database seeding)
└── prisma/
    └── schema.prisma (already created)
```

Critical Implementation Notes

1. Challenge Generation with AI

When generating challenges, use the Foto ChatLLM API:

```
const response = await fetch('https://apps.abacus.ai/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.ABACUSAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4.1-mini',
    messages: [{
      role: 'system',
      content: 'You are Foto, an expert photography coach...'
    }, {
      role: 'user',
      content: `Generate a ${challengeType} challenge for ${context}...`
    }],
    response_format: { type: "json_object" },
    max_tokens: 1500
  })
});
```

2. Email Templates

Use the HTML email templates provided in Section 8 of the spec. Implement for:
- Saturday challenge email (with PDF itinerary)
- Lunch break micro-challenge email
- WFH flexible challenge email
- Golden hour alert
- Weekly progress report

3. Weather API Integration

```
// Get weather
const response = await fetch(
  `https://api.openweathermap.org/data/2.5/forecast?lat=${lat}&lon=${lon}&appid=${API_
KEY}&units=imperial`
);

// Check conditions
const condition = data.list[0].weather[0].main; // "Clear", "Rain", "Fog"
const temp = data.list[0].main.temp;
```

4. Scheduled Tasks

Use the `perform_subtask_daemon_management` tool or implement using Next.js API routes with external cron service (Vercel Cron, etc.).

5. Progress Tracking

Implement the analytics functions from Section 12:
- `calculateCompletionRate()`
- `adaptDifficulty()`
- `analyzeSkillProgression()`
- `calculateTrend()`

Expected Output

After implementation, the system should:

1. ✅ User can sign up and complete onboarding
2. ✅ User receives appropriate challenges based on day/context
3. ✅ User can mark challenges complete and upload photos
4. ✅ Progress stats update in real-time
5. ✅ Scheduled tasks run automatically
6. ✅ Emails send successfully
7. ✅ Weather integration works
8. ✅ Dashboard displays all relevant information
9. ✅ Settings page allows full customization
10. ✅ System adapts to user behavior over time

Testing Checklist

Before considering implementation complete, verify:

- [ ] Signup/login flow works
- [ ] Onboarding wizard completes
- [ ] Dashboard loads with sample data
- [ ] Can generate Saturday challenge manually
- [ ] Can generate weekday lunch challenge
- [ ] Challenge can be marked complete
- [ ] Photo upload works
- [ ] Progress stats calculate correctly
- [ ] Settings save and apply
- [ ] Email sends successfully (test mode)
- [ ] Weather API returns data
- [ ] Scheduled tasks can be triggered manually
- [ ] All pages are responsive
- [ ] No console errors

Final Deployment

Once implementation is complete:

1. Run `test_nextjs_project` to verify no errors
2. Run `build_and_save_nextjs_project_checkpoint`
3. Deploy to production
4. Set up scheduled tasks in production environment
5. Verify email delivery in production
6. Monitor agent action logs

Questions to Ask Me

If you need clarification during implementation:

1. **About the specification**: Ask me to explain any section
2. **About design choices**: Show me your approach and ask for feedback
3. **About external services**: If you need API keys or credentials
4. **About testing**: If you want me to test specific functionality
5. **About deployment**: If you need help with production setup

Reference Documents

- **Main Spec**: `FOTO_AI_AGENT_IMPLEMENTATION_SPEC.md` (attached)
- **Current App**: `/home/ubuntu/foto_app/nextjs_space/`
- **Database**: Already initialized and schema pushed
- **Deployment URL**: https://myfoto.topues.com

Start Implementation

Please follow this approach:

1. **Read the entire specification document first**
2. **Create a task list** (use the phases from the spec)
3. **Start with Phase 1** (Database seeding and authentication)
4. **Implement incrementally** and test each phase
5. **Ask questions** if anything is unclear
6. **Show progress** after each major milestone

Begin implementation now. Start with Phase 1 and work through systematically. Good luck! 🚀📸

PROMPT END

How to Use This Prompt

1. **Copy** the entire prompt above (from “PROMPT START” to “PROMPT END”)
2. **Attach** the file `FOTO_AI_AGENT_IMPLEMENTATION_SPEC.md`
3. **Paste** into your AI assistant (ChatGPT, Claude, Gemini, etc.)
4. **Start** the implementation

The AI will have everything needed to build the complete Foto AI Agent system.